



服务于大规模应用程序的键值存储的开发优先级的演变： RocksDB 经验

Siying Dong、Andrew Kryczka 和 Yanqin Jin, *Facebook Inc.*; Michael Stumm, *University of Toronto* <https://www.usenix.org/conference/fast21/presentation/dong>

本文收录在第 19 届 USENIX 文件和存储技术会议的会议记录中。 February 23-25, 2021

978-1-939133-20-5

第 19 届 USENIX 文件和存储技术会议论文集的开放获取由 USENIX 赞助。

服务于大规模应用程序的键值存储的开发优先级的演变：RocksDB 经验

Siying Dong[†]、Andrew Kryczka[†]、Yanqin Jin[†] 和 Michael Stumm[‡]

[†] Facebook Inc., 1 Hacker Way, 门洛帕克, 加利福尼亚州, 美国
[‡] 多伦多大学, 加拿大多伦多

抽象的

RocksDB 是一个针对大规模分布式系统的键值存储, 并针对固态硬盘 (SSD) 进行了优化。本文描述了过去八年里我们开发 RocksDB 的优先事项是如何演变的。这种演变是硬件趋势和许多组织在生产中大规模运行 RocksDB 的丰富经验的结果。我们描述了 RocksDB 的资源优化目标如何以及为何从写入放大迁移到空间放大, 再到 CPU 利用率。运行大型应用程序的经验教训告诉我们, 需要跨不同的 RocksDB 实例管理资源分配, 数据格式需要保持向后和向前兼容以允许增量软件部署, 并且需要对数据库复制和备份提供适当的支持。故障处理的经验教训告诉我们, 需要在系统的每一层更早地检测到数据损坏错误。

1 简介

RocksDB [19, 54] 是 Facebook 在 2012 年基于 Google 的 LevelDB 代码库 [22] 创建的高性能持久键值存储引擎。它针对固态硬盘 (SSD) 的特定特性进行了优化, 面向大规模 (分布式) 应用程序, 并被设计为嵌入到更高级别应用程序中的库组件。因此, 每个 RocksDB 实例只管理单个服务器节点的存储设备上的数据; 它不处理任何主机间操作, 例如复制和负载平衡, 也不执行高级操作, 例如检查点——它将这些操作的实现留给应用程序, 但提供适当的支持, 以便它们能够有效地完成这些操作。

RocksDB 及其各种组件是高度可定制的, 允许存储引擎根据广泛的需求和工作负载进行定制; 定制可以包括预写日志 (WAL) 处理、压缩策略和压缩策略 (一个过程)

删除死数据并优化 LSM 树, 如 §2) 中所述。RocksDB 可以针对高写入吞吐量或高读取吞吐量、空间效率或介于两者之间的某些目标进行调整。由于其可配置性, RocksDB 被许多应用程序使用, 代表了广泛的用例。仅在 Facebook, RocksDB 就被 30 多个不同的应用程序使用, 总共存储了数百 PB 的生产数据。除了用作 MySQL [37]、Rocksandra [6]、CockroachDB [64]、MongoDB [40] 和 TiDB [27] 的存储引擎外, RocksDB 还用于以下具有高度不同特征的服务类型 (表 1 中总结):

- **Stream processing**: RocksDB 用于在 Apache Flink [12]、Kafka Stream [31]、Samza [43] 和 Facebook 的 Stylus [15] 中存储临时数据。
- **Logging/queuing services**: Facebook 的 LogDevice [5] (同时使用 SSD 和 HDD)、Uber 的 Cherami [8] 和 Iron.io [29] 使用 RocksDB。
- **Index services**: Facebook 的 Dragon [59] 和 Rockset [58] 使用 RocksDB。
- **Caching on SSD**: 内存缓存服务, 例如 Netflix 的 E VCache [7]、奇虎的 Pika [51] 和 Redis [46], 使用 RocksDB 将从 DRAM 移出的数据存储存储在 SSD 上。

之前的一篇文章使用 RocksDB [11] 分析了几种数据库应用程序。表 2 总结了从生产工作负载中获得的一些关键系统指标。

拥有可以支持许多不同用例的存储引擎的优点是在不同的应用程序中使用相同的存储引擎。事实上, 让每个应用程序构建自己的存储子系统是有问题的, 因为这样做具有挑战性。即使是简单的应用程序也需要使用校验和来防止介质损坏, 保证崩溃后的数据一致性, 以正确的顺序发出正确的系统调用以保证写入的持久性, 并以正确的方式处理从文件系统返回的错误。完善的通用存储引擎可以在所有这些领域提供复杂性。

当客户端应用程序在一个计算机中运行时, 通过使用通用存储引擎可以获得额外的好处。

	Read/Write	Read Types	Special Characteristics
Databases	Mixed	Get + Iterator	Transactions and backups
Stream Processing	Write-Heavy	Get or Iterator	Time window and checkpoints
Logging / Queues	Write-Heavy	Iterator	Support HDD too
Index Services	Read-Heavy	Iterator	Bulk loading
Cache	Write-Heavy	Get	Can drop data

表 1: RocksDB 用例及其工作负载特征

	CPU	Space Util	Flash Endurance	Read Bandwidth
Stream Processing	11%	48%	16%	1.6%
Logging / Queues	46%	45%	7%	1.0%
Index Services	47%	61%	5%	10.0%
Cache	3%	78%	74%	3.5%

表 2: 每个应用程序类别的典型用例的系统指标。

mon 基础设施：监控框架、性能分析工具和调试工具都可以共享。例如，公司内的不同应用程序所有者可以利用相同的内部框架，将统计数据报告到相同的仪表板，使用相同的工具监控系统，并使用相同的嵌入式管理服务管理 RocksDB。这种整合不仅允许不同团队之间轻松地重用专业知识，还允许将信息聚合到公共门户，并鼓励开发工具来管理它们。

鉴于采用 RocksDB 的应用程序多种多样，其开发优先级自然也发生了变化。本文描述了过去八年中我们的优先事项是如何演变的，因为我们从现实世界的应用程序（Facebook 和其他组织内部）中吸取了实践教训，并观察到了硬件趋势的变化，使我们重新审视了一些早期的假设。我们还描述了近期 RocksDB 的开发重点。

§2 提供了 SSD 和日志结构合并 (LSM) 树的背景 [45]。RocksDB 从一开始就选择 LSM 树作为其主要数据结构，以解决读/写性能不对称以及基于闪存的 SSD 的有限耐用性。我们相信 LSM 树已经为 RocksDB 提供了很好的服务，并认为即使面对即将到来的硬件趋势，它们也将仍然非常适合 (§3)。LSM 树数据结构是 RocksDB 能够适应具有不同需求的不同类型应用程序的原因之一。

§3 描述了我们的主要优化目标如何从最小化写入放大转向最小化空间放大，以及从优化性能转向优化效率。

§4 描述了我们在服务大规模分布式系统方面学到的经验教训；例如：(i) 资源分配必须

跨多个 RocksDB 实例进行管理，因为单个服务器可以托管多个实例；(ii) 数据格式必须向后和向前兼容，因为 RocksDB 软件更新是增量部署/回滚的；(iii) 对数据库复制和备份的适当支持非常重要。

§5 描述了我们处理故障的经验。大型分布式系统通常使用复制来实现容错和高可用性。然而，必须正确处理单节点故障才能实现该目标。我们发现仅仅识别和传播文件系统和校验和错误是不够的。相反，必须尽早识别每一层的错误（例如位翻转），并且应用程序应该能够在可能的情况下指定以自动方式处理这些错误的策略。

§6 展示了我们对改进键值接口的想法。虽然核心接口简单而强大，但由于其灵活性，它限制了一些关键用例的性能。我们描述了对与键和值分开的用户定义时间戳的支持。

§8 列出了 RocksDB 将从未来研究中受益的几个领域。

2 背景

Flash 的特性深刻地影响了 RocksDB 的设计。读/写性能的不对称和有限的耐用性给数据结构和系统架构的设计带来了挑战和机遇。因此，RocksDB 采用闪存友好的数据结构并针对现代硬件进行了优化。

2.1 基于闪存的 SSD 嵌入式存储

在过去的十年中，我们目睹了基于闪存的 SSD 用于在线数据服务的激增。低延迟和高吞吐量的设备不仅要求软件充分利用其功能，而且还改变了有状态服务的实施数量。SSD 每秒可提供数十万次的读取和写入输入/输出操作 (IOPS)，这比旋转硬盘驱动器快数千倍。它还可以支持数百 MB 的带宽。然而，由于编程/擦除周期数量有限，无法维持高写入带宽。这些因素提供了重新思考存储引擎的数据结构以针对该硬件进行优化的机会。

SSD 的高性能在很多情况下也将延迟和吞吐量的性能瓶颈从设备 I/O 转移到了网络。对于应用程序来说，将其架构设计为将数据存储在本地的 SSD 上而不是使用远程数据存储服务变得更具吸引力。这增加了对嵌入应用程序中的键值存储引擎的需求。

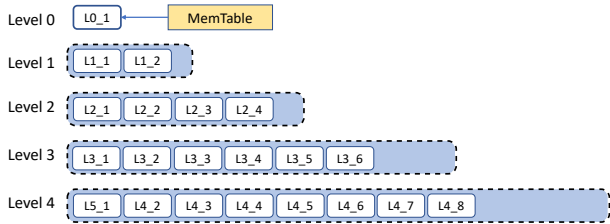


图 1：使用分层压缩的 RocksDB LSM 树。每个白框都是一个 SStable。

RocksDB 的创建就是为了满足这些需求。我们希望创建一个灵活的键值存储来为使用本地 SSD 驱动器的各种应用程序提供服务，同时针对 SSD 的特性进行优化。LSM 树在实现这些目标中发挥了关键作用。

2.2 RocksDB 架构

RocksDB 使用日志结构合并（LSM）树[45]作为其存储数据的主要数据结构。

Writes. 每当数据写入 RocksDB 时，它都会添加到名为 *MemTable* 的内存写入缓冲区以及磁盘上的预写日志（WAL）中。Memtable 被实现为跳表，因此可以通过 $O(\log n)$ 插入和搜索开销保持数据有序。WAL 用于故障后的恢复，但不是强制的。Once the size of the MemTable reaches a configured size, then (i) the MemTable and WAL become immutable, (ii) a new MemTable and WAL are allocated for subsequent writes, (iii) the contents of the MemTable are *flushed* to a “Sorted String Table” (SStable) data file on disk, and (iv) the flushed MemTable and associated WAL are *dis-梳理*。每个 SStable 按排序顺序存储数据，分为大小统一的块。每个 SStable 还具有一个索引块，每个 SStable 块有一个索引条目用于二分搜索。

Compaction. LSM 树有多个级别的 SStables，如图 1 所示。最新的 SStables 由 MemTable 刷新创建并放置在 Level-0 中。高于 Level-0 的级别由名为 *compaction* 的进程创建。给定级别上 SStable 的大小受到配置参数的限制。当超过 level-L 的大小目标时，level-L 中的一些 SStable 会被选择并与 level-(L+1) 中重叠的 SStable 合并。这样做时，删除和覆盖的数据将被删除，并且表将针对读取性能和空间效率进行优化。此过程逐渐将写入的数据从 Level-0 迁移到最后一层。压缩 I/O 非常高效，因为它可以并行化，并且只涉及整个文件的批量读取和写入。

0 级 SStable 具有重叠的键范围，因为每个 SStable 涵盖完整排序的运行。后面的级别每个仅包含一个排序运行，因此这些级别中的 SStable 包含其级别的排序运行的一个分区。

Reads. 在读取路径中，在每个连续级别都会发生键查找，直到找到该键或确定该键已存在。

Compaction	W A	M O	A O	G	m	G	r	k
Leveled	16.07	9.8%	9.5%	0.99	1.7	1.84		
Tiered	4.8	94.4%	45.5%	1.03	3.39	4.80		
FIFO	2.14	N/A	N/A	1.16	528	967		

表 3：RocksDB 5.9 下三种主要压缩类型的写入放大、开销和读取 I/O。对于分层压缩，排序运行数设置为 12，每个键 20 个布隆过滤器位用于 FIFO 压缩。使用直接 I/O，并将块缓存大小设置为完全压缩的数据库大小的 10%。写入放大的计算方式为 SStable 文件写入总量与刷新的 Mem-Table 字节数。不包括 WAL 写入。

最后一级中不存在密钥。它首先搜索所有 MemTable，然后搜索所有 Level-0 SStable，然后依次搜索更高级别的 SStable。在每个级别，都使用二分搜索。布隆过滤器用于消除 SStable 文件中不必要的搜索。扫描要求搜索所有级别。

RocksDB 支持多种不同类型的压缩。

Leveled Compaction 改编自 LevelDB，然后改进[19]。在这种压缩方式中，级别被分配成指数增加的大小目标，如图 1 中的虚线框所示。RocksDB 中称为 *Universal Compaction* 的 *Tiered Compaction*（与 Apache Cassandra 或 HBase 使用的类似。当排序运行太多，或者总数据库大小与最大排序运行大小之间的比率超过可配置阈值时，多个排序运行会延迟压缩在一起。最后，一旦数据库达到大小限制，*FIFO Compaction* 就会简单地丢弃旧文件，并且只执行轻量级压缩。它针对内存缓存应用程序。

能够配置压缩类型使 RocksDB 能够服务于广泛的应用例。通过使用不同的压缩方式，RocksDB 可以针对特殊的缓存工作负载配置为读友好、写友好或非写友好。然而，应用程序所有者需要考虑其特定用例的不同指标之间的权衡[2]。惰性压缩算法可以提高写入放大和写入吞吐量，但读取性能会受到影响，而更积极的压缩会牺牲写入性能，但可以实现更快的读取速度。日志记录或流处理等服务可以使用大量写入的设置，而数据库服务则需要平衡的方法。表 3 通过微基准测试结果描述了这种灵活性。

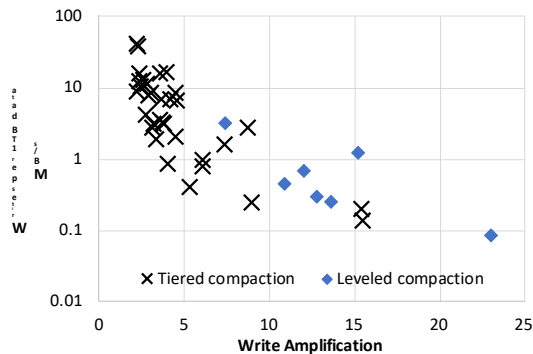


图 2: 对 42 个随机采样的 ZippyDB 和 MyRocks 应用程序的写入放大和写入速率的调查。

3 资源优化目标的演变

在这里，我们描述了我们的资源优化目标如何随着时间的推移而演变：从写入放大到空间放大再到CPU利用率。

写放大

当我们开始开发 RocksDB 时，我们最初关注的是节省闪存擦除周期，从而实现写入放大，遵循当时社区的普遍观点（例如，[34]）。对于许多应用程序来说，这确实是一个重要目标，特别是对于那些具有大量写入工作负载的应用程序（表 1），这仍然是一个问题。

写放大出现在两个层面上。SSD 本身引入了写放大：根据我们在 1.1 和 3 之间的观察。存储和数据库软件也会产生写放大；有时该值可能高达 100（例如，当为小于 100 字节的更改写出整个 4KB/8KB/16KB 页面时）。

RocksDB 中的分级压缩通常会表现出 10 到 30 之间的写入放大，这在许多情况下比使用 B 树要好几倍。例如，当在 MySQL 上运行 LinkBench 时，RocksDB 每个事务发出的写入次数仅为 InnoDB（一种基于 B 树的存储引擎）的 5% [37]。尽管如此，10-30 范围内的写入放大对于写入密集型应用来说还是太高了。出于这个原因，我们添加了分层压缩，它将写入放大降低到 4-10 范围，但读取性能较低；参见表 3。图 2 描述了 RocksDB 在不同数据摄取速率下的写入放大情况。RocksDB 应用程序所有者通常会在写入速率较高时选择压缩方法来减少写入放大，并在写入速率较低时更积极地进行压缩，以实现空间效率和读取性能目标。

空间放大

经过几年的开发，我们观察到，对于大多数应用程序来说，空间利用率比写入放大重要得多，因为闪存写入周期和写入开销都不受限制。事实上，与 SSD 相比，实际使用的 IOPS 数量较低

	Dynamic Leveled Compaction			LevelDB-style Compaction		
	compaction frequency	B-tree	SSD	compaction frequency	B-tree	SSD
200	12.0	13.5	12.4	12.0	15.1	25.6
400	24.0	26.9	11.8	24.0	26.9	12.2
600	36.0	40.4	12.2	36.4	42.5	16.9
800	48.0	54.2	12.7	48.3	57.9	19.7
1,000	60.1	67.5	12.4	60.3	73.8	22.4

表 4: 在微基准测试中测量的 RocksDB 空间效率：数据已预先填充，每次写入都是从预先填充的密钥空间中随机选择的密钥。RocksDB 5.9 具有所有默认选项。恒定 2MB/s 写入速率。

提供（但仍然高得足以使 HDD 失去吸引力，即使忽略维护开销）。因此，我们将资源优化目标转移到磁盘空间。

幸运的是，LSM 树由于其非碎片数据布局，在优化磁盘空间时也能很好地工作。然而，我们看到了通过减少 LSM 树中的死数据（即删除和覆盖的数据）量来改进 Leveled Compaction 的机会。我们开发了

Dynamic Leveled Compaction，其中树中每个级别的大小根据最后一个级别的实际大小自动调整（而不是静态设置每个级别的大小）[19]。该方法比 Leveled Compaction 具有更好、更稳定的空间效率。表 3 显示了随机写入基准测试中测得的空间效率：动态分级压缩将空间开销限制为 13%，而分级压缩可以增加 25% 以上。此外，Leveled Compaction 下最坏情况下的空间开销可高达 90%，而对于动态 Leveling 则比较稳定。事实上，对于 Facebook 主要数据库之一 UDB 来说，当 InnoDB 被 RocksDB 取代后，空间占用减少到了 50% [36]。

CPU 利用率

有时会引起人们关注的一个问题是，SSD 的速度变得如此之快，以至于软件无法再充分利用其潜力。也就是说，对于 SSD 来说，瓶颈已经从存储设备转移到了 CPU，因此需要对软件进行根本性的改进。根据我们的经验，我们并不认同这种担忧，并且我们预计它不会成为未来基于 NAND 闪存的 SSD 的问题，原因有两个。首先，只有少数应用程序受到 SSD 提供的 IOPS 的限制；正如第 4.2 节中所讨论的，大多数应用都受到空间的限制。

其次，我们发现任何配备高端 CPU 的服务器都具有足够的计算能力来满足一个高端 SSD 的需求。RocksDB 在我们的环境中充分利用 SSD 性能从未遇到过问题。当然，可以配置一个系统，使 CPU 成为一个

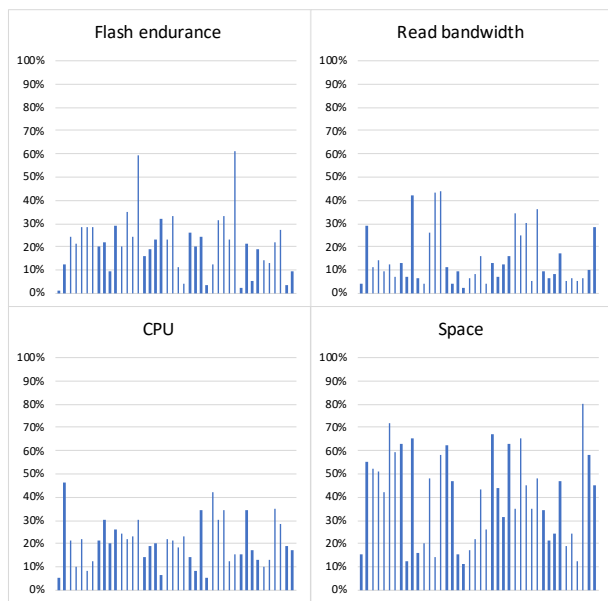


图 3: 四个指标的资源利用率。每条线代表具有不同工作负载的不同部署。测量是在一个月的时间内进行的。所有数字都是部署中所有主机的平均值。CPU 和读取带宽是该月中最高的时间。闪存耐用性和空间利用率是整个月的平均水平。

瓶颈;例如, 具有一个 CPU 和多个 SSD 的系统。然而, 有效的系统通常是那些配置良好平衡的系统, 这是当今技术所允许的。以写入为主的密集工作负载也可能导致 CPU 成为瓶颈。对于某些人来说, 可以通过将 RocksDB 配置为使用更轻量级的压缩选项来缓解这一问题。对于其他情况, 工作负载可能根本不适合 SSD, 因为它会超出允许 SSD 持续 2-5 年的典型闪存耐用性预算。

为了证实我们的观点, 我们调查了生产中 ZippyDB [65] 和 MyRocks 的 42 种不同部署, 每种部署服务于不同的应用程序。图 3 显示了结果。大多数工作负载都受到空间限制。有些确实占用大量 CPU, 但主机通常没有得到充分利用, 无法为增长和处理数据中心或区域级故障 (或由于配置错误) 留出空间。这些部署中的大多数包括数百台主机, 因此考虑到工作负载可以在这些主机之间自由 (重新) 平衡, 平均值可以了解这些用例的资源需求 (§4)。

尽管如此, 鉴于减少空间放大这一唾手可得的成果已经获得, 减少 CPU 开销已经成为一个重要的优化目标。减少 CPU 开销可以提高 CPU 确实受到限制的少数应用程序的性能。更重要的是, 减少 CPU 开销的优化允许更具成本效益的硬件配置 —

直到几年前, CPU 和内存的价格相对于 SSD 来说还相当低, 但 CPU 和内存的价格已大幅上涨, 因此降低 CPU 开销和内存使用量变得越来越重要。降低 CPU 开销的早期努力包括引入前缀布隆过滤器、在索引查找之前应用布隆过滤器以及其他布隆过滤器改进。仍有进一步改进的空间。

适应新技术

与 SSD 相关的新架构改进可能很容易破坏 RocksDB 的相关性。例如, 开放通道 SSD [50, 66]、多流 SSD [68] 和 ZNS [4] 有望改善查询延迟并节省闪存擦除周期。然而, 这些新技术只会让少数使用 RocksDB 的应用程序受益, 因为大多数应用程序都受到空间限制, 而不是擦除周期或延迟受到限制。此外, 让 RocksDB 直接适应这些技术将对统一的 RocksDB 体验提出挑战。一个值得探索的可能路径是将这些技术的容纳委托给底层文件系统, 或许 RocksDB 会提供额外的提示。

存储内计算可能会带来显着的收益, 但尚不清楚有多少 RocksDB 应用程序实际上会从中受益。我们怀疑 RocksDB 适应存储内计算将具有挑战性, 可能需要对整个软件堆栈进行 API 更改才能充分利用。我们期待未来研究如何最好地做到这一点。

分解 (远程) 存储似乎是一个更有趣的优化目标, 也是当前的优先事项。到目前为止, 我们的优化假设闪存是本地连接的, 因为我们的系统基础设施主要是这样配置的。然而, 更快的网络目前允许远程提供更多的 I/O, 因此使用远程存储运行 RocksDB 的性能对于越来越多的应用程序来说已经变得可行。通过远程存储, 可以更轻松地同时充分利用 CPU 和 SSD 资源, 因为它们可以按需单独配置 (这对于本地连接的 SSD 来说更难实现)。因此, 针对远程闪存存储优化 RocksDB 已成为当务之急。我们目前正在通过尝试整合和并行化 I/O 来解决长 I/O 延迟的挑战。我们采用 RocksDB 来处理瞬时故障, 将 QoS 要求传递给底层系统, 并报告分析信息。然而, 还需要做更多的工作。

存储级内存 (SCM) 是一项很有前途的技术。我们正在研究如何最好地利用它。有几种可能性值得考虑: 1. 使用 SCM 作为 DRAM 的扩展——这提出了如何最好地使用混合 DRAM 和 SCM 实现关键数据结构 (例如块缓存或内存表) 的问题, 以及在尝试利用所提供的持久性时会引入哪些开销; 2. 使用 SCM 作为数据库的主存储, 但我们注意到 RocksDB 往往会受到空间或 CPU 的瓶颈, 而不是

比 I/O；3. 对 WAL 使用 SCM，但这提出了一个问题：考虑到在将其转移到 SSD 之前我们只需要一个小的暂存区域，这个用例本身是否就证明了 SCM 的成本是合理的。

重温主要数据结构

我们不断地重新审视 LSM 树是否仍然合适的问题，但不断得出它们仍然合适的结论。SSD 的价格还没有下降到足以改变大多数用例的空间和闪存耐用性瓶颈的程度，而用 CPU 或 DRAM 来交换 SSD 使用率的替代方案仅对少数应用程序有意义。虽然主要结论保持不变，但我们经常听到用户对写入放大的需求低于 RocksDB 所能提供的。然而，我们注意到，当对象大小很大时，可以通过分离键和值来减少写入放大（例如 WiscKey [35] 和 ForestDB [1]），因此我们将其添加到 RocksDB（称为 BlobDB）。

为大型系统提供服务的 4 个经验教训

RocksDB 是具有不同需求的各种大型分布式系统的构建块。随着时间的推移，我们了解到资源管理、WAL 处理、批量文件删除、数据格式兼容性和配置管理方面需要改进。

资源管理

大规模分布式数据服务通常将数据划分为 *shards*，分布在多个服务器节点上进行存储。分片的大小是有限的，因为分片是负载平衡和复制的单位，并且因为分片为此目的在节点之间自动复制。因此，每个服务器节点通常会托管数十或数百个分片。在我们的上下文中，一个单独的 RocksDB 实例用于为每个分片提供服务，这意味着存储主机上将运行许多 RocksDB 实例。这些实例可以全部运行在一个地址空间中，也可以各自运行在自己的地址空间中。

主机可能运行许多 RocksDB 实例这一事实对资源管理有影响。鉴于实例共享主机的资源，需要对资源进行全局（每个主机）和本地（每个实例）管理，以确保公平、高效地使用它们。当在单进程模式下运行时，全局资源限制很重要，包括（1）写入缓冲区和块缓存的内存，（2）压缩 I/O 带宽，（3）压缩线程，（4）总磁盘使用量和（5）文件删除率（如下所述），并且基于每个 I/O 设备可能需要这样的限制。还需要本地资源限制，例如确保单个实例不能使用过量的任何资源。RocksDB 允许应用程序为每种类型的资源创建一个或多个资源控制器（实现为传递给不同 DB 对象的 C++ 对象），并且还可以在每个实例的基础上执行此操作。最后，重要的是

支持 RocksDB 实例之间的优先级划分，以确保资源优先分配给最需要的实例。

在一个进程中运行多个实例时学到的另一个教训是：大量生成非池化线程可能会出现，特别是当线程寿命较长时。线程过多会增加 CPU 的概率，导致过多的上下文切换开销，并使调试变得极其困难，并且会导致 I/O 峰值。如果 RocksDB 实例需要使用可能进入睡眠或等待条件的线程执行某些工作，那么最好使用可以轻松限制大小和资源使用的线程池。

当 RocksDB 实例在单独的进程中运行时，全局（每主机）资源管理更具挑战性，因为每个分片仅具有本地信息。可以应用两种策略。首先，每个实例都配置为保守地使用资源，而不是贪婪地使用资源。例如，通过压缩，每个实例可以启动比“正常”更少的压缩，只有当压缩落后时才会增加。这种策略的缺点是全局资源可能没有得到充分利用，导致资源利用率不高。第二种在操作上更具挑战性的策略是让实例在它们之间共享资源使用信息并相应地进行调整，以尝试更全局地优化资源使用。需要做更多的工作来改进 RocksDB 中主机范围的资源管理。

水处理

传统数据库倾向于在每次写入操作时强制执行预写日志 (WAL) 写入，以确保持久性。相比之下，大规模分布式存储系统通常会为了性能和可用性而复制数据，并且这样做时会提供各种一致性保证。例如，如果相同数据的副本存在于多个副本中，并且其中一个副本损坏或无法访问，则存储系统将使用来自其他未受影响的主机的有效副本来重建故障主机的副本。对于此类系统，RocksDB WAL 写入不太重要。此外，分布式系统通常有自己的复制日志（例如 Paxos 日志），在这种情况下根本不需要 RocksDB WAL。

我们了解到，提供调整 WAL 同步行为的选项以满足不同应用程序的需求很有帮助。具体来说，我们引入了不同的 WAL 操作模式：(i) 同步 WAL 写入、(i) 缓冲 WAL 写入和 (i) 根本不写入 WAL。对于缓冲 WAL 处理，WAL 会在后台以低优先级定期写入磁盘，以免影响 RocksDB 的流量延迟。

限速文件删除

RocksDB 通常通过文件系统与底层存储设备交互。这些文件系统支持闪存 SSD；例如，每当删除文件时，具有实时丢弃功能的 XFS 可能会向 SSD 发出 TRIM 命令 [28]。人们普遍认为 TRIM 命令可以提高性能和闪存耐用性 [21]，我们的生产经验也证实了这一点。但是，它也可能导致性能问题。修剪

比我们最初想象的更具破坏性：除了更新地址映射（通常在 SSD 的内部存储器中）之外，SSD 固件还需要将这些更改写入闪存中的 FTL¹ 日志，这反过来可能会触发 SSD 的内部垃圾收集，导致大量数据移动，从而对前台 IO 延迟产生负面影响。为了避免 TRIM 活动峰值和相关的 I/O 延迟增加，我们引入了文件删除速率限制，以防止同时删除多个文件（这发生在压缩之后）。

数据格式兼容性

大型分布式应用程序在许多主机上运行其服务，并且期望零停机时间。因此，软件升级会在主机上逐步推出；当出现问题时，更新会回滚。鉴于持续部署[56]，这些软件升级频繁发生；RocksDB 每月发布一次新版本。因此，磁盘上的数据在不同软件版本之间保持向后和向前兼容非常重要。新升级（或回滚）的 RocksDB 实例必须能够理解先前实例存储在磁盘上的数据。此外，RocksDB 数据文件可能需要在分布式实例之间复制以进行副本构建或负载平衡，并且这些实例可能运行不同的版本。缺乏前向兼容性保证导致某些 RocksDB 部署中出现操作困难，这导致我们添加了保证。

RocksDB 竭尽全力确保数据保持向前和向后兼容（新功能除外）。这在技术和流程方面都具有挑战性，但我们发现付出的努力得到了回报。为了向后兼容，RocksDB 必须能够理解之前写入磁盘的所有格式；这增加了软件和维护的复杂性。为了向前兼容，需要了解未来的数据格式，我们的目标是保持向前兼容性至少一年。这可以通过使用通用技术来部分实现，例如 Protocol Buffer [63] 或 Thrift [62] 使用的技术。对于配置文件条目，RocksDB 需要能够识别新字段，并尽力猜测如何应用配置或何时丢弃。我们用不同版本的数据不断测试不同版本的 RocksDB。

管理配置

RocksDB 具有高度可配置性，因此应用程序可以针对其工作负载进行优化。然而，我们发现配置管理是一个挑战。最初，RocksDB 继承了 LevelDB 的参数配置方法，参数选项直接嵌入代码中。这导致了两个问题。首先，参数选项通常与存储在磁盘上的数据相关联，当使用一个选项创建的数据文件无法由新配置的 RocksDB 实例打开另一个选项时，会导致潜在的兼容性问题。

¹FTL: Flash Translation Layer.

Config Area:	Compaction	I/O	Compression	SSTable file	Plug-in fcts
Configurations:	14	4	2	7	6

表 5: 39 个 ZippyDB 部署中使用的不同配置的数量

选项。其次，代码中未明确指定的配置选项会自动设置为 RocksDB 的默认值。当 RocksDB 软件更新包括对默认配置参数的更改（例如，增加内存使用或压缩并行性）时，应用程序有时会遇到意想不到的后果。

为了解决这些问题，RocksDB 首先引入了 RocksDB 实例使用包含配置选项的字符串参数打开数据库的功能。后来 RocksDB 引入了对选择性地将选项文件与数据库一起存储的支持。我们还引入了两个工具：(i) 一个验证工具，用于验证打开数据库的选项是否与目标数据库兼容；(ii) 迁移工具重写数据库以与所需选项兼容（尽管此工具有限制）。

RocksDB 配置管理的一个更严重的问题是大量的配置选项。在 RocksDB 的早期，我们做出了支持高度定制的设计选择：我们引入了许多新的旋钮，并引入了可插组件的支持，所有这些都是为了让应用程序发挥其性能潜力。事实证明，这是一个在早期获得初步吸引力的成功策略。然而，现在常见的抱怨是选择太多，而且很难理解它们的效果；也就是说，指定“最佳”配置变得非常困难。

除了需要调整许多配置参数之外，更令人畏惧的是，最佳配置不仅取决于嵌入 RocksDB 的系统，还取决于其之上的应用程序生成的工作负载。例如，考虑 ZippyDB [65]，这是一种内部开发的大规模分布式键值存储，在其节点上使用 RocksDB。ZippyDB 为许多不同的应用程序提供服务，有时是单独的，有时是多租户设置。尽管我们付出了巨大的努力，尽可能在所有 ZippyDB 用例中使用统一的配置，但不同用例的工作负载差异很大，当性能很重要时，统一的配置实际上是不可行的。表 5 显示，在我们抽样的 39 个 ZippyDB 部署中，有超过 25 个不同的配置。

对于交付给第三方的嵌入式 RocksDB 系统来说，调整配置参数也特别具有挑战性。考虑第三方在其应用程序之一中使用 MySQL 或 ZippyDB 等数据库。第三方通常对 RocksDB 及其最佳调整方式知之甚少。而且数据库所有者对调整客户的系统没有兴趣。

这些现实世界的教训引发了我们配置支持策略的变化。我们在提高开箱即用性能和简化配置方面投入了大量精力。鉴于 RocksDB 继续为专门的应用程序提供服务，我们当前的重点是提供 *automatic adaptivity*，同时继续支持广泛的显式配置。我们注意到，在保留显式可配置性的同时追求适应性会产生大量的代码维护开销，我们相信拥有统一存储引擎的好处超过了代码复杂性。

复制和备份支持

RocksDB 是一个单节点库。如果需要，使用 RocksDB 的应用程序负责复制和备份。每个应用程序都以自己的方式实现这些功能（出于合法原因），因此 RocksDB 为这些功能提供适当的支持非常重要。

通过复制现有副本的所有数据来引导新副本可以通过两种方式完成。首先，可以从源副本读取所有密钥，然后将其写入目标副本（*logical copying*）。在源端，RocksDB 通过提供最小化对并发在线查询影响的能力来支持数据扫描操作；例如，通过提供不缓存这些操作结果的选项，从而防止缓存垃圾。在目的地端，支持批量加载，并针对此场景进行了优化。

其次，可以通过直接复制 SSTable 和其他文件来引导新的副本（*physical copying*）。RocksDB 通过识别当前时间点的现有数据库文件并防止它们被删除或变异来协助物理复制。支持物理复制是 RocksDB 将数据存储于底层文件系统上的一个重要原因，因为它允许每个应用程序使用自己的工具。我们相信 RocksDB 直接使用块设备接口或与 FTL 深度集成的潜在性能提升不会超过上述好处。

备份对于大多数数据库和其他应用程序来说是一项重要功能。对于备份，应用程序具有与复制相同的逻辑与物理选择。备份和复制之间的区别之一是应用程序通常需要管理多个备份。虽然大多数应用程序实现自己的备份（以满足自己的要求），但 RocksDB 提供了一个备份引擎供应用程序在备份要求简单时使用。

我们认为该领域有两个需要进一步改进的领域，但都需要对键值 API 进行更改；它们在第 6 节中讨论。第一个涉及在不同副本上以一致的顺序应用更新，这带来了性能挑战。第二个问题涉及一次发出一个写入请求的性能问题，以及副本可能落后的事实，而应用程序可能希望这些副本能够更快地赶上。不同的应用程序已经实施了各种解决方案来解决这些问题，但它们都有局限性[20]。挑战在于应用程序不能乱序写入并执行快照

使用自己的序列号读取，因为 RocksDB 目前不支持具有用户定义时间戳的多版本控制。

故障处理的 5 个教训

通过生产经验，我们学到了故障处理的三大教训。首先，需要尽早检测到数据损坏，以最大限度地降低数据不可用或丢失的风险，并查明错误的根源。其次，完整性保护必须覆盖整个系统，以防止静默损坏暴露给 RocksDB 客户端或传播到其他副本（见图 4）。三是要区别对待错误。

无声腐败的频率

出于性能原因，RocksDB 用户通常不会使用 SSD（例如 DIF/DIX）进行数据保护，并且存储介质损坏是通过 RocksDB 块校验和来检测的，这是所有成熟数据库的常规功能，因此我们在此跳过分析。CPU/内存损坏很少发生，而且很难准确量化。使用 RocksDB 的应用程序通常会运行数据一致性检查来比较副本的完整性。这会捕获错误，但这些错误可能是由 RocksDB 或客户端应用程序引入的（例如，在复制、备份或恢复数据时）。

我们发现，在 RocksDB 级别引入的损坏频率可以通过比较具有这两种索引的 MyRocks 数据库表中的主索引和辅助索引来估计；RocksDB 级别引入的任何不一致，包括 CPU 或内存损坏。根据我们的测量，RocksDB 级别的每 100PB 数据大约每三个月就会出现一次损坏。更糟糕的是，在其中 40% 的情况下，损坏已经传播到其他副本。

传输数据时也会发生数据损坏，通常是由于软件错误造成的。例如，在处理网络故障时，底层存储系统中的一个错误导致我们在一段时间内看到，传输的每 PB 物理数据大约有 17 个校验和不匹配。

多层保护

需要尽早检测到数据损坏，以最大限度地减少停机时间和数据丢失。大多数 RocksDB 应用程序都将数据复制到多个主机上；当检测到校验和不匹配时，损坏的副本将被丢弃并用正确的副本替换。然而，只有当正确的副本仍然存在时，这才是一个可行的选择。

如今，RocksDB 对多个级别的文件数据进行校验，以识别其下方层中的损坏。这些以及计划的应用程序层校验和如图 4 所示。多层校验和很重要，主要是因为它们有助于及早检测损坏，并且可以防止不同类型的威胁。块校验和，继承

从 LevelDB 中，防止文件系统或文件系统以下损坏的数据暴露给客户端。2020 年添加的文件校验和可防止底层存储系统传播到其他副本而导致的损坏，以及通过线路传输 SSTable 文件时导致的损坏。对于 WAL 文件，切换校验和可以在写入时有效地及早检测损坏。

Block integrity. 每个 SSTable 块或 WAL 片段都附有一个校验和，该校验和是在创建数据时生成的。与仅在移动文件时验证的文件校验和不同，由于其范围较小，每次读取数据时都会验证该校验和。这样做可以防止存储层损坏的数据暴露给 RocksDB 客户端。

File integrity. 文件内容在传输操作过程中尤其面临被损坏的风险；例如，用于备份或分发 SSTable 文件时。为了解决这个问题，SSTables 受到它们自己的校验和的保护，校验和是在创建表时生成的。SSTable 的校验和记录在元数据的 SSTable 文件条目中，并且无论传输到何处，都会使用 SSTable 文件进行验证。然而，我们注意到其他文件，例如 WAL 文件，仍然没有受到这种方式的保护。

Handoff integrity. 早期检测写入损坏的一种既定技术是在要写入底层文件系统的数据上生成切换校验和，并将其与数据一起向下传递，由较低层进行验证[48, 70]。我们希望使用这样的写入 API 来保护 WAL 写入，因为与 SSTables 不同，WAL 受益于每个追加的增量验证。不幸的是，本地文件系统很少支持这一点——但是一些专门的堆栈，例如 Oracle ASM [49] 却支持这一点。

另一方面，当在远程存储上运行时，可以更改写入 API 以接受校验和，挂钩到存储服务的内部 ECC。RocksDB 可以在现有的 WAL 片段校验和上使用校验和组合技术来有效地计算写切换校验和。由于我们的存储服务执行写入时验证，因此我们预计损坏检测很少会延迟到读取时。

端到端保护

虽然上述保护层在许多情况下可以防止客户端暴露于损坏的数据，但它们并不全面。到目前为止提到的保护措施的一个缺陷是文件 I/O 层之上的数据不受保护。例如，MemTable 和块缓存中的数据。在此级别损坏的数据将无法检测到，因此最终将暴露给用户。此外，刷新或压缩操作可以保留损坏的数据，使损坏永久化。

Key-value integrity. 为了解决这个问题，我们目前正在实施每个键值校验和来检测文件 I/O 层之上发生的损坏。无论复制到哪里，该校验和都将与键/值一起传输，尽管我们将从已经存在替代完整性保护的文件数据中删除它。

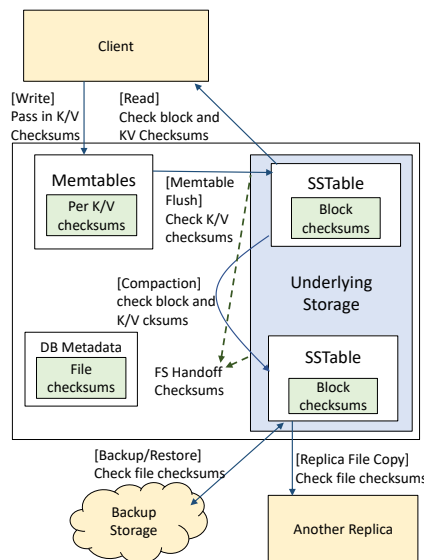


图 4：四种类型的校验和

基于严重性的错误处理

RocksDB 遇到的故障大部分是底层存储系统返回的错误。这些错误可能源于多种问题，从只读文件系统等严重问题到磁盘已满或访问远程存储的网络错误等暂时性问题。早期，RocksDB 对此类问题的反应要么是简单地向客户端返回错误消息，要么永久停止所有写入操作。

今天，我们的目标是仅当错误不可本地恢复时才中断 RocksDB 操作；例如，暂时性网络错误不应需要用户干预来重新启动 RocksDB 实例。为了实现这一点，我们改进了 RocksDB，在遇到暂时性错误后定期重试恢复操作。因此，我们获得了运营优势，因为客户不需要针对发生的大部分故障手动缓解 RocksDB。

关于键值接口的 6 课

核心键值 (KV) 接口的通用性令人惊讶。几乎所有存储工作负载都可以由具有 KV API 的数据存储提供服务；我们很少看到应用程序无法使用此接口实现功能。这也许就是 KV 商店如此受欢迎的原因。KV 接口是通用的。键和值都是可变长度字节数组。应用程序可以非常灵活地确定将哪些信息打包到每个键和值中，并且可以从一组丰富的编码方案中自由选择。因此，应用程序负责解析和解释键和值。KV 接口的另一个好处是它的可移植性。从一种键值系统迁移到另一种键值系统相对容易。然而，虽然许多用例通过这个简单的界面实现了最佳性能，但我们注意到

它可能会限制某些应用程序的性能。

例如，在 RocksDB 之外构建并发控制是可能的，但很难提高效率，特别是如果需要支持两阶段提交（在提交事务之前需要一些数据持久性）。为此，我们添加了事务支持，MyRocks (MySQL+RocksDB) 使用了该事务支持。我们不断添加功能；例如，间隙/下一个键锁定和大事务支持。

在其他情况下，限制是由键值接口本身引起的。因此，我们开始研究基本键值接口的可能扩展。其中一项扩展是支持用户定义的时间戳。

版本和时间戳

在过去的几年里，我们逐渐认识到数据版本控制的重要性。我们得出的结论是，版本信息应该成为 RocksDB 中的一等公民，以便正确支持多版本并发控制 (MVCC) 和时间点读取等功能。为了实现这一点，RocksDB 需要能够有效地访问不同的版本。

到目前为止，RocksDB 内部一直使用 56 位序列号来识别不同版本的 KV 对。序列号由 RocksDB 生成，并在每次客户端写入时递增（因此，所有数据都按排序顺序逻辑排列）。客户端应用程序不能影响序列号。然而，RocksDB 允许应用程序获取数据库的 *Snapshot*，之后 RocksDB 保证快照时存在的所有 KV 对将持续存在，直到应用程序显式释放快照为止。因此，具有相同密钥的多个 KV 对可能会共存，并通过它们的序列号进行区分。

这种版本控制方法是不够的，因为它不能满足许多应用程序的要求。要读取过去的状态，必须已在前一个时间点拍摄快照。RocksDB 不支持拍摄过去的快照，因为没有 API 来指定时间点。此外，支持时间点读取的效率很低。最后，每个 RocksDB 实例分配自己的序列号，并且只能在每个实例的基础上获取快照。这使得具有多个（可能是复制的）分片的应用程序的版本控制变得复杂，每个分片都是一个 RocksDB 实例。总之，创建提供跨分片一致读取的数据版本基本上是不可能的。

应用程序可以通过在键或值内对时间戳进行编码来解决这些限制。然而，在任何一种情况下，它们都会经历性能下降。在键内编码会牺牲点查找的性能，而在值内编码会牺牲对同一键进行无序写入的性能，并使旧版本键的读取变得复杂。我们相信应用程序指定的时间戳可以更好地解决这些限制，应用程序可以使用可以全局理解的时间戳来标记其数据，并且在键或值之外执行此操作。

我们添加了对特定应用程序的基本支持

workload	throughput gain
fill_seq + read_random	1.2
fill_seq + read_while_writing	1.9
fill_random + read_random	1.9
fill_random + read_while_writing	2.0

表 6：使用时间戳 API 的 DB_bench 微基准测试发现 ≥ 吞吐量提高了 1.2 倍。

时间戳并使用 DB-Bench 评估了这种方法。结果如表 6 所示。每个工作负载都有两个步骤：第一步填充数据库，第二步测量性能。例如，在“fill_seq + read_random”中，我们通过按升序写入多个键来填充初始数据库，并在步骤 2 中执行随机读取操作。相对于基线，应用程序将时间戳编码为密钥的一部分（对 RocksDB 透明），应用程序指定的时间戳 API 可以带来 1.2 倍或更好的吞吐量增益。这些改进源于将时间戳视为与用户键分开的元数据，因为这样可以使用点查找而不是迭代器来获取键的最新值，并且布隆过滤器可以识别不包含该键的 SSTable。此外，SSTable 涵盖的时间戳范围可以存储在其属性中，可以利用该属性来排除只能包含过时值的 SSTable。

我们希望此功能能够让用户更轻松地在其系统中实现多版本控制，以实现单节点 MVCC、分布式事务或解决多主复制中的冲突。然而，更复杂的 API 使用起来不太简单，而且可能容易被误用。此外，与不存储时间戳相比，数据库会消耗更多的磁盘空间，并且对其他系统的可移植性较差。

7 相关工作

我们在 RocksDB 上的工作受益于多个领域的广泛研究。

存储引擎库

许多存储引擎已被构建为嵌入到应用程序中的库。RocksDB 的 KV 接口比 BerkeleyDB [44]、SQLite [47] 和 Hekaton [18] 等更原始。此外，RocksDB 与这些系统的不同之处在于，它专注于现代服务器工作负载的性能，这些工作负载需要高吞吐量和低延迟，并且通常在高端 SSD 和多核 CPU 上运行。这与具有更通用目标的系统不同，或者是为更快的存储介质构建的系统 [18, 30]。

SSD 的键值存储

多年来，人们在优化键值存储方面付出了很多努力，尤其是对于 SSD。早在 2011 年，SILT [34] 就提出了一种在内存效率、CPU 和性能之间取得平衡的键值存储。ForestDB [45] 使用 HB+

在日志顶部建立索引的树。TokuDB [32] 和其他数据库使用 FractalTree/B+ 树。LOCS [67]、NoFTL-KV [66] 和 FlashKV [69] 针对开放通道 SSD 以提高性能。虽然 RocksDB 从这些努力中受益，但我们提高性能的立场和策略有所不同，我们继续依赖 LSM 树。一些研究将 RocksDB 与其他数据库（例如 InnoDB [41]、TokuDB [19] [37] 和 WiredTiger [10]）的性能进行了比较。

LSM 树改进

一些系统还使用 LSM 树并提高了它们的性能。写放大通常是主要优化目标；例如，WiscKey [35]、PebblesDB [52]、IAM-tree [25] 和 TRIAD [3]。这些系统在优化写入放大方面比 RocksDB 更进一步，RocksDB 更注重不同指标之间的权衡。SlimDB [53] 优化了 LSM 树以提高空间效率；RocksDB 还专注于删除死数据。Monkey [17] 试图在 DRAM 和 IOP 之间取得平衡。bLSM [57]、VT-tree [60] 和 cLSM [24] 针对 LSM 树的一般性能进行了优化。

大型存储系统

有许多分布式存储系统[13,14,16,26,38,64]。它们通常具有跨越多个进程、主机和数据中心的复杂架构。它们不能直接与单节点上的存储引擎库 RocksDB 相比较。其他系统（例如，MongoDB、MySQL [42]、Microsoft SQL Server [38]）可以使用模块化存储引擎；他们解决了与 RocksDB 面临的类似挑战，包括故障处理和使用时间戳。

Failure handling. 校验和经常用于检测数据损坏[9,23,42]。我们需要端到端和切换校验和的论点仍然反映了经典的端到端论点[55]，并且类似于其他人使用的策略：[61]、ZFS [71]、Linux [48] 和 [70]。我们关于早期腐败检测的论点类似于[33]，后者认为特定领域的检查是不够的。

Timestamp support. 一些存储系统提供时间戳支持：HBase [26]、WiredTiger [39] 和 BigTable [14]；Cassandra [13] 支持将时间戳作为普通列。在这些系统中，时间戳是自 UNIX 纪元以来的毫秒数的计数。Hekaton [18] 使用单调递增的计数器来分配时间戳，这与 RocksDB 序列号类似。RocksDB 在用户时间戳方面正在进行的工作可以对上述工作进行补充。我们希望具有用户定义的时间戳扩展的键值 API 可以使上层系统更容易支持与数据版本控制相关的功能，同时在性能和效率方面的开销较低。

8 未来的工作和悬而未决的问题

除了完成上面提到的改进之外，包括针对分解存储、键值分离的优化

配给、多级校验和和应用程序指定的时间戳，我们计划统一分层和分层压缩并提高适应性。然而，一些悬而未决的问题可能会从进一步的研究中受益。

1. 如何使用 SSD/HDD 混合存储来提高效率？
2. 当存在多个连续删除标记时，如何减轻对读取器的性能影响？
3. 我们应该如何改进我们的写限制算法？
4. 我们能否开发一种有效的方法来比较两个副本以确保它们包含相同的数据？
5. 我们如何才能最好地利用 SCM？我们是否还应该使用 LSM 树以及如何组织存储层次结构？
6. 是否可以有一个通用的完整性 API 来处理 RocksDB 和文件系统层之间的数据切换？

9 结论

RocksDB 已经从服务于利基应用程序的键值存储发展到目前在众多工业大型分布式应用程序中广泛采用的地位。LSM 树作为主要数据结构，很好地服务于 RocksDB，因为它表现出良好的写入和空间放大能力。然而，我们对性能的看法随着时间的推移而发生了变化。虽然写入和空间放大仍然是主要问题，但额外的重点已转移到 CPU 和 DRAM 效率以及远程存储。

运行大型应用程序的经验教训告诉我们，需要跨不同的 RocksDB 实例管理资源分配，数据格式需要保持向后 *and forward* 兼容以允许增量软件部署，需要对数据库复制和备份提供适当的支持，并且配置管理需要简单且最好是自动化的。故障处理的经验教训告诉我们，需要在系统的每一层更早地检测到数据损坏错误。键值接口因其简单性而广受欢迎，但在性能上存在一些限制。对界面进行一些简单的修改可能会产生更好的平衡。

致谢

我们将 RocksDB 的成功归功于 Facebook 所有现任和过去的 RocksDB 团队成员、所有在开源社区做出贡献的人以及 RocksDB 用户。我们特别感谢该项目多年来的导师 Mark Callaghan，以及 RocksDB 的主要创始成员 Dhruba Borthakur。我们还感谢 Jason Flinn 和 Mahesh Balakrishnan 对本文的评论。最后，我们感谢我们的牧羊人伊森·米勒 (Ethan Miller) 和匿名审稿人的宝贵反馈。

RocksDB 功能时间表

	Performance	Configurability	Features
2 1 0 2	Multi-threaded compactions		- Compaction filters - Locking SSTables from deletion
3 0 1 2	- Tiered compaction - Prefix Bloom filter - Bloom Filter for MemTables - Separate thread pool for MemTable flush	- Pluggable MemTable - Pluggable file format	Merge Operator
4 1 0 2	- FIFO compaction - Compaction rate limiter - Cache-friendly Bloom filters	- String-based config options - Dynamic config changes	- Backup engine - Support for multiple key spaces ("column family") - Physical checkpoints
5 1 0 2	- Dynamic leveled compaction - File deletion rate limiting - Parallel Level 0 and 1 compaction	- Separate config file - Config compatibility checker	- Bulk loading for SSTable file integration - Optimistic and pessimistic transactions
6 1 0 2	- Different compression for last level - Parallel MemTable inserts	- MemTable total size caps across instances - Compaction migration tools	DeleteRange()
7 1 0 2	- Separate thread pool for bottom-most compactions - Two-level file indices - Level 0 to level 0 compactions	Single memory limit for both block cache and MemTable	
8 1 0 2	- Dictionary compression - Hash index into data blocks		- Automatic recovery from out-of-space errors - Query trace and replay tools
9 1 0 2	Batched MultiGet() with parallel I/O	Configure plug-in function using object registry	Secondary instance
0 2 0 2	Multithreaded single file compression		- Entire file checksum - Automatically recover from retrievable errors - Partial support for user-defined timestamps

B 回顾经验教训

我们吸取的一些教训包括：

1. 可以调整存储引擎以适应不同的性能特征，这一点很重要。（§1）
2. 空间效率是大多数使用 SSD 的应用程序的瓶颈。（§3，空间放大）
3. 为了让系统更高效地运行，CPU 开销变得越来越重要。（§3，CPU 利用率）
4. 当多个 RocksDB 实例运行在同一主机上时，需要对每台主机进行全局资源管理。（§4，资源管理）有效值
5. 让 WAL 处理可配置（同步 WAL 写入、缓冲 WAL 写入或禁用 WAL）提供了应用程序系统蒸发性能优势。（§4，WAL 处理）钙-
6. SSD TRIM 操作对性能有好处，但需要限制文件删除的速率，以防止偶尔出现性能问题。（§4，限速文件删除）CE
7. RocksDB 需要提供向后和“向前”的兼容性。（§4，数据格式兼容性）
8. 自动配置自适应有助于简化配置管理。（§4，管理配置）
9. 需要适当支持数据复制和备份。（§4，复制和备份支持）
10. 尽早发现数据损坏比最终发现更有利。（§5）
11. CPU/内存损坏确实会发生，尽管很少见，而且有时无法通过数据复制来处理。（§5）
12. 完整性保护必须覆盖整个系统，以防止损坏的数据（例如，由 CPU/内存中的位翻转引起）暴露给客户端或其他副本；仅当数据处于静止状态或通过线路发送时检测损坏是不够的。（§5）
13. 用户经常要求 RocksDB 自动从瞬时 I/O 错误中恢复，例如空间不足或由网络问题引起。（§5）
14. 需要根据错误的原因和后果以不同的方式对待错误处理。（§5）
15. key/value 接口通用性强，但存在一定的性能限制；向键/值添加时间戳可以提供性能和简单性之间的良好平衡。（§6）拉

C 重新审视设计选择的回顾

重新审视的一些值得注意的设计选择包括：

1. 可定制性对用户来说总是有好处的。（§4，管理配置）
2. RocksDB 可能对 CPU 位翻转视而不见。（§5）
3. 看到任何 I/O 错误时出现恐慌是正常的。（§5）

参考

- [1] Jung-Sang Ahn, Chiyoung Seo, Ravi Mayuram, Rahim Yaseen, Jin-Soo Kim 和 Seungryoul Maeng. Forest DB: 用于可变长度字符串键的快速键值存储系统。 *IEEE Trans. on Computers*, 65(3): 902–915, 2015.
- [2] Manos Athanassoulis, Michael S Kester, Lukas M Maas, Radu Stoica, Stratos Idreos, Anastasia Ailamaki 和 Mark Callaghan. 设计访问方法: RUM 猜想。 *Proc. Intl. Conf on Extending Database Technology (EDBT)*, 2016 年卷, 第 461–466 页, 2016 年。
- [3] Oana Balmau, Diego Didona, Rachid Guerraoui, Willy Zwaenepoel, Huapeng Yuan, Aashray Arora, Karan Gupta 和 Pavan Konka. TRIAD: 在日志结构的键值存储中创建内存、磁盘和日志之间的协同作用。 *Proc. USENIX Annual Technical Conference (USENIX-ATC'17)*, 第 363–375 页, 2017 年。
- [4] 马蒂亚斯·比约林. 区域追加: 一种写入分区存储的新方法。在 *Proc. Usenix Linux Storage and Filesystems Conference (VAULT'20)*, 2020 年。
- [5] Facebook 工程博客。LogDevice: 日志的分布式数据存储。 <https://engineering.fb.com/core-data/logdevice-a-distributed-data-store-for-logs/>。[在线的;检索于 2020 年 9 月]。
- [6] Instagram 工程博客。开源后, Apache Cassandra 尾部延迟减少了 10 倍。 <https://instagram-engineering.com/open-source-a-10x-reduction-in-apache-cassandra-tail-latency-d64f86b43589>。[在线的;检索于 2020 年 9 月]。
- [7] Netflix 技术博客。使用 SSD 进行应用程序数据缓存: Moneta 项目: 下一代 EV 缓存, 可实现更好的成本优化。 <https://netflixtechblog.com/application-data-caching-protect-discretionary-hyphenchar-font-ussing-ssds-5bf25df851ef>。[在线的; 2020 年 9 月检索]。
- [8] 优步工程博客。Cherami: Uber Engineering 在 Go 中的持久且可扩展的任务队列。 <https://eng.uber.com/cherami-message-queue-system/>。[在线的;检索于 2020 年 9 月]。
- [9] 德鲁巴·博塔库尔。HDFS 架构指南。 *Hadoop Apache Project*, 53(1-13):2, 2008。
- [10] 马克·卡拉汉。MongoRocks 和 WiredTiger 与小型服务器上的 LinkBench。 <http://smallldatum.blogspot.com/2016/10/mongorocks-and-wiredtiger-versus.html>。[在线的;检索于 2021 年 1 月]。
- [11] 曹志超, 董思英, Sagar Vemuri, David H.C.杜。对 Facebook 的 RocksDB 键值工作负载进行表征、建模和基准测试。 *18th USENIX Conf. on File and Storage Technologies (FAST'20)*, 第 209–223 页, 2020 年 2 月。
- [12] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi 和 Kostas Tzoumas. Apache Flink: 在单个引擎中进行流处理和批处理。 *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 36(4), 2015。
- [13] 阿帕奇卡桑德拉。 <https://cassandra.apache.org/>。[在线的;检索于 2020 年 9 月]。
- [14] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C Hsieh, Deborah A Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes 和 Robert E Gruber. Bigtable: 结构化数据的分布式存储系统。 *ACM Trans. on Computer Systems (TOCS)*, 26(2):1–26, 2008。
- [15] Guoqiang Jerry Chen, Janet L Wiener, Shridhar Iyer, Anshul Jaiswal, Ran Lei, Nikhil Simha, Wei Wang, Kevin Wilfong, Tim Williamson 和 Serhat Yilmaz. Facebook 的实时数据处理。 *Proc. Intl. Conf. on Management of Data*, 第 1087–1098 页, 2016 年。
- [16] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild 等。Spanner: Google 的全球分布式数据库。 *ACM Trans. on Computer Systems (TOCS)*, 31(3):1–22, 2013。
- [17] Niv Dayan, Manos Athanassoulis 和 Stratos Idreos. Monkey: 最佳可导航键值存储。 *Proc. Intl. Conf. on Management of Data (SIGMOD'17)*, 第 79–94 页, 2017 年。
- [18] 克里斯蒂安·迪亚科努、克雷格·弗里德曼、埃里克·伊斯默特、佩尔·阿克·拉尔森、普拉文·米塔尔、瑞安·斯通西弗、尼廷·维尔马和迈克·兹威林。Hekaton: SQL Server 的内存优化 OLTP 引擎。 *Proc. ACM SIGMOD Intl. Conf. on Management of Data (SIGMOD'13)*, 第 1243–1254 页, 2013 年。
- [19] Siying Dong, Mark Callaghan, Leonidas Galanis, Dhruba Borthakur, Tony Savor 和 Michael Stumm. 优化 RocksDB 中的空间放大。在 *Proc. Conf. on Innovative Data Systems Research (CIDR'17)*, 2017 年。
- [20] 何塞·法莱罗。逻辑复制的危险和实用的解决方案。在 *Proc. 18th Intl. Workshop on High Performance Transaction Systems (HPTS'19)*, 2019 年。

- [21] 塔莎·弗兰基、戈登·休斯和肯·克罗伊茨-德尔加多。NAND 闪存 SSD 中修剪命令的数学模型。 *Proc. 50th Annual Southeast Regional Conference (ACM-SE'12)*, 第 59-64 页, 2012 年。
- [22] S. Ghemawat 和 J. Dean。级别数据库。 <https://github.com/google/leveldb>, 2011。
- [23] Sanjay Ghemawat、Howard Gobioff 和 Shun-Tak Leung。谷歌文件系统。见 *Proc. 19th ACM Symp. on Operating systems principles (SOSP'13)*, 第 29-43 页, 2003 年。
- [24] Guy Golan-Gueta、Edward Bortnikov、Eshcar Hillel 和 Idit Keidar。扩展并发日志结构数据存储。 *Proc. European Conf. on Computer Systems (EUROSYS'15)*, 2015 年第 1-14 页。
- [25] 龚财新, 何水兵, 龚伊丽, 雷迎春。关于日志结构合并树中追加和合并的集成。 *Proc. 48th Intl. Conf. on Parallel Processing (ICPP'19)*, 2019 年第 1-10 页。
- [26] 阿帕奇 HBase。 <https://hbase.apache.org/>。[在线的; 检索于 2020 年 9 月]。
- [27] 黄东旭, 刘奇, 崔秋, 方柱河, 马晓宇, 徐飞, 沉力, 刘唐, 周宇兴, 黄猛龙, 万伟, 刘丛, 张建, 李建军, 吴雪莲, 宋凌宇, 孙若曦, 于帅鹏, 赵雷, 尼古拉斯·卡梅伦, 裴利泉, 唐鑫。TiDB: 基于 Raft 的 HTAP 数据库。 *Proc. VLDB Endow.*, 13(12):3072–3084, 2020 年 8 月。
- [28] 英特尔。修剪概览。 <https://www.intel.com/content/www/us/en/support/articles/000016148/memory-and-storage.html>。[在线的; 检索于 2021 年 1 月]。
- [29] 铁.io。Confluence <https://www.iron.io>。[在线的; 检索于 2020 年 9 月]。
- [30] 木村秀明。FOEDUS: 用于一千个核心和 NVRAM 的 OLTP 引擎。 *Proc. SIGMOD Intl. Conf. on Management of Data (SIGMOD'15)*, 第 691-706 页, 2015 年。
- [31] 杰·克雷普斯。Kafka Streams 简介: 流处理变得简单。Confluent <https://www.confluent.io/blog/introducing-kafka-streams-stream-processing-made-simple/>。[在线的; 检索于 2020 年 9 月]。
- [32] B·库兹莫尔。TokuDB 分形树索引如何工作。技术报告, 技术报告, TokuTek, 2010 年。
- [33] 查克·杠杆。NFS 的端到端数据完整性要求。甲骨文公司 <https://datatracker.ietf.org/conference/83/material/slides/83-nfsv4-2>。[在线的; 检索于 2020 年 9 月]。
- [34] Hyeontaek Lim、Bin Fan、David G Andersen 和 Michael Kaminsky。SILT: 内存高效、高性能的键值存储。 *Proc. 23rd ACM Symp. on Operating Systems Principles (SOSP'11)*, 第 1-13 页, 2011 年。
- [35] Lanyue Lu、Thanumalayan Sankaranarayanan Pillai、Hariharan Gopalakrishnan、Andrea C Arpaci-Dusseau 和 Rami H Arpaci-Dusseau。Wiskey: 将 SSD 敏感存储中的键与值分开。 *ACM Trans. on Storage (TOS)*, 13(1): 1–28, 2017。
- [36] 松信义德。将数据库从 InnoDB 迁移到 MyRock。Facebook 工程博客 <https://engineering.fb.com/core-data/migration-a-database-from-innodb-to-myrocks/>, 2017 年。[在线; 检索于 2020 年 9 月]。
- [37] 松信义典, 董思英, 赫尔曼·李。MyRocks: 服务于 Facebook 社交图谱的 LSM 树数据库存储引擎。 *Proc. VLDB Endowment*, 13(12): 3217–3230, 2020 年 8 月。
- [38] 微软。微软 SQL 服务器。 <https://www.microsoft.com/en-us/sql-server/>。[在线的; 检索于 2020 年 9 月]。
- [39] MongoDB。WiredTiger 存储引擎。 <https://docs.mongodb.com/manual/core/wiredtiger/>。[在线的; 检索于 2020 年 9 月]。
- [40] 蒙戈洛克斯。MongoDB 的 RocksDB 存储引擎模块。 <https://github.com/mongodb-partners/mongo-rocks>。[在线的; 检索于 2020 年 9 月]。
- [41] MySQL。InnoDB 简介。 <https://dev.mysql.com/doc/refman/5.6/en/innodb-introduction.html>。[在线的; 检索于 2020 年 9 月]。
- [42] MySQL。MySQL。 <https://www.mysql.com/>。[在线的; 检索于 2020 年 9 月]。
- [43] Shadi A Noghabi, Kartik Paramasivam, Yi Pan, Navina Ramesh, Jon Bringham, Indranil Gupta, and Roy H Campbell。Samza: Stateful scalable stream processing at LinkedIn。 *Proc. of the VLDB Endowment*, 10 (12) : 1634-1645, 2017。
- [44] Michael A Olson, Keith Bostic, and Margo I Seltzer。伯克利数据库。In *USENIX Annual Technical Conference, FREENIX Track*, pages 183–191, 1999。
- [45] 帕特里克·奥尼尔、爱德华·程、迪特·高利克和伊丽莎白·奥尼尔。The log-structured merge-tree (LSM-tree)。 *Acta Informatica*, 33(4):351–385, 1996。

- [46] Keren Ouaknine, Oran Agra and Zvika Guz. RocksDB for Redis on flash 的优化。 *Proc. Intl. Conf. on Compute and Data Analysis*, 第 155-161 页, 2017 年。
- [47] 迈克·欧文斯。 *The definitive guide to SQLite*。阿普莱斯, 2006。
- [48] 马丁·K 彼得森。Linux 数据完整性扩展。在 *Linux Symposium*, 第 4 卷, 第 5 页, 2008 年。
- [49] 马丁·K 彼得森和塞尔吉奥·洛尼森。使用 Oracle Linux 消除静默数据损坏。甲骨文公司 <https://oss.oracle.com/~mkp/docs/data-integrity-webcast.pdf>。[在线的;检索于 2020 年 9 月]。
- [50] Ivan Luiz Picoli, Niclas Hedam, Philippe Bonnet, and Pinar Tözün. Open-channel SSD (What is it good for). In *Proc. Conf. on Innovative Data Systems Research (CIDR'20)*, 2020. [51] Qihoo. Confluent <https://github.com/Qihoo360/pika>。[在线的;检索于 2020 年 9 月]。[52] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. PebblesDB: Building key-value stores using fragmented log-structured merge trees. In *Proc. 26th Symp. on Operating Systems Principles (SOSP'17)*, pages 497–514, 2017. [53] Kai Ren, Qing Zheng, Joy Arulraj, and Garth Gibson. SlimDB: A space-efficient key-value storage engine for semi-sorted data. *Proc. of the VLDB Endowment (VLDB'17)*, 10(13):2037–2048, 2017. [54] RocksDB.org. A persistent key-value store for fast storage environments. <https://rocksdb.org>。[在线的;检索于 2020 年 9 月]。[55] Jerome H Saltzer, David P Reed, and David D Clark. End-to-end arguments in system design. *ACM Trans. on Computer Systems (TOCS)*, 2(4):277–288, 1984. [56] Tony Savor, Mitchell Douglas, Michael Gentili, Laurie Williams, Kent Beck, and Michael Stumm. Continuous deployment at Facebook and OANDA. In *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 21–30. IEEE, 2016. [57] Russell Sears and Raghu Ramakrishnan. bLSM: a general purpose log-structured merge tree. In *Proc. Intl. Conf. on Management of Data (SIGMOD'12)*, pages 217–228, 2012. [58] Arun Sharma. How we use RocksDB at Rockset. Rockset Blog <https://rockset.com/blog/how-we-use-rocksdb-at-rockset/>。[在线的;检索于 2020 年 9 月]。
- [59] 阿伦·夏尔马。LogDevice: 日志的分布式数据存储。Facebook Engineering Blog <https://engineering.fb.com/data-infrastructure/dragon-a-distributed-graph-query-engine/>。[在线的;检索于 2020 年 9 月]。[60] Pradeep J Shetty, Richard P Spillane, Ravikant R Malpani, Binesh Andrews, Justin Seyster, and Erez Zadok. 使用 VT 树构建独立于工作负载的存储。In *Proc. 11th USENIX Conf. on File and Storage Technologies (FAST'13)*, pages 17–30, 2013. [61] Gopalan Sivan, Charles P Wright, and Erez Zadok. Enhancing file system integrity through checksums. Technical report, Citeseer, 2004. [62] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. Thrift: Scalable cross-language services implementation. *Facebook White Paper*, 5(8), 2007. [63] Google Open Source. 原型缓冲区。 <https://opensource.google/projects/protobuf>。[在线的;检索于 2020 年 9 月]。[64] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VandenBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. CockroachDB: The resilient geo-distributed SQL database. In *Proc. ACM SIGMOD Intl. Conf. on Management of Data (SIGMOD'20)*, page 1493–1509, 2020. [65] Amy Tai, Andrew Kryczka, Shobhit O. Kanaujia, Kyle Jamieson, Michael J. Freedman, and Asaf Cidon. Who's afraid of uncorrectable bit errors? Online recovery of flash errors with distributed redundancy. In *2019 USENIX Annual Technical Conference (USENIX ATC'19)*, pages 977–992, Renton, WA, July 2019. [66] Tobias Vinçon, Sergej Hardock, Christian Riegger, Julian Oppermann, Andreas Koch, and Ilia Petrov. NoFTL-KV: Tackling write-amplification on KV-stores with native storage management. In *Proc. 21st Intl. Conf. on Extending Database Technology (EDBT'18)*, 第 457-460 页, 2018 年。[67] 王鹏、孙光宇、宋江、欧阳剑、林世鼎、张晨和丛杰。An efficient design and implementation of LSM-tree based key-value store on open-channel SSD. 见 *Proc. 9th European Conf. on Computer Systems (EUROSYS'14)*, 第 1-14 页, 2014 年。[68] Fei Yang, K Dou, S Chen, JU Kang and S Cho. Multi-streaming RocksDB. 在 *Proc. Non-Volatile Memories Workshop*, 2015 年。

[69] Jiacheng Zhang, Youyou Lu, Jiwu Shu, and Xiongjun Qin. FlashKV: 使用开放通道 SSD 加速 KV 性能。 *ACM Trans on Embedded Computing Systems (TECS)*, 16 (5s): 1-19, 2017. [70] 张玉璞, 丹尼尔·S·迈尔斯, 安德里亚·C·阿尔帕奇-杜索, 雷姆兹·H·阿尔帕奇-杜索。字节可靠性和灵活的端到端数据完整性。见 *Proc. 29th IEEE Symp. on Mass Storage Systems and Technologies (MSST'13)*, 第 1-14 页, 2013 年。 [71] 张玉璞、Abhishek Rajimwale、Andrea C Arpaci-Dusseau 和 Remzi H Arpaci-Dusseau。文件系统的端到端数据完整性: ZFS 案例研究。 *Proc. 8th USENIX Conf. on File and Storage Technologies (FAST'10)*, 第 29-42 页, 2010 年。